

Turf — Build Brief (Learning Edition)

A web app to find a game to play when you don't have a team. Hosts post a game that's short on players; anyone nearby can see it and grab a slot. Around that core, this build deliberately includes a full set of backend features.

Goal of this project: practice FastAPI end to end — auth, relationships, concurrency, websockets, background tasks, third-party integrations, geospatial queries, RBAC, testing, migrations, Docker. Scope is intentionally broad. Build it **module by module**, each as its own vertical slice, and finish + commit one before starting the next.

1. Tech stack (fixed)

- **Frontend:** React + Vite + Tailwind (kept simple — this is a backend project)
 - **Backend:** FastAPI + SQLAlchemy
 - **Database:** Postgres (+ PostGIS for the geolocation module)
 - **Migrations:** Alembic — a migration for every schema change, from day one
 - **Cache / queue:** Redis (used by notifications, caching, rate limiting)
 - **Auth:** `fastapi-users` (JWT) + role-based scopes
 - **Testing:** pytest + httpx `AsyncClient`
 - **Infra:** Docker + docker-compose (api, postgres, redis)
 - Separate `/frontend` and `/backend` apps in one repo; CORS enabled for the Vite origin.
-

2. Data model

Start from `models.py` (users, games, game_players). The schema **grows per module** — each module below lists the tables it adds. Rules that hold throughout:

- Open slots on a game are **derived** from `len(game.players)`, never stored as a counter.
 - The host auto-joins as player 1 on game creation.
 - `slots_filled` / `slots_open` are computed in a mapper, not DB columns.
 - Every schema change ships with an Alembic migration.
-

3. Modules

Each module is a vertical slice (models → migration → endpoints → tests → minimal UI). The 🎓 line is the point of the module — the backend concept you're practicing.

Module 0 — Project setup

Both apps, Postgres + Redis via docker-compose, settings via Pydantic (`BaseSettings`), `/health` endpoint, Alembic initialised, pytest wired, one seeded dev user. 🎓 *Project structure, env config, dependency injection* (`get_db`), *migrations baseline, test harness*.

Module 1 — Auth & users

Register, login (JWT), password hashing, `get_current_user` dependency, user profile (name, skill_level, city, role). 🎓 *JWT auth, password hashing, dependency-injected auth*, `fastapi-users`. Adds: `role` field on `users` (used later by RBAC).

Module 2 — Games core loop

`POST /games` (host auto-joins), `GET /games` (filter by sport/city/status, paginated), `GET /games/{id}`, `POST /games/{id}/join`. Join edge cases: game not found → 404; not `open` → 409; already joined → 409; on fill, flip status `open → full` (in the join handler only). 🎓 *CRUD, query-param filtering, pagination, relationships, computed response fields, transactional join, constraint handling, a status state machine*.

Module 3 — Venues & inventory

CRUD for venues; a game can reference a venue. Nested listing (`GET /venues/{id}/games`). 🎓 *One-to-many relationships, nested resources, pagination, fuller CRUD*. Adds: `venues` (name, address, city, sports, hourly_rate, opening/closing hours, lat, lng).

Module 4 — Bookings (the concurrency module)

Book a venue for a time slot. Two users must not book the same venue+slot. 🎓 *DB concurrency — prevent double-booking with a unique constraint and row locking* (`SELECT ... FOR UPDATE`) *inside a transaction; time-overlap queries; pessimistic vs optimistic locking. This is the most important backend lesson in the project*. Adds: `bookings` (venue_id, user_id, starts_at, ends_at, status).

Module 5 — Payments

Charge for a booking via **Razorpay** (Indian gateway). Create order → client pays → gateway calls your webhook → you verify the signature and mark the booking paid. 🎓 *Third-party API integration, webhooks, webhook signature verification, idempotency keys, handling out-of-order/duplicate callbacks*. Adds: `payments` (booking_id, amount, currency, gateway_order_id, status).

Module 6 — Real-time chat

A chat per game over WebSockets; messages persist. 🎓 *FastAPI WebSockets, a connection manager, broadcasting, authenticating a WS connection, persisting + replaying history*. Adds:

`messages` (game_id, user_id, body, created_at).

Module 7 — Ratings & karma

Players rate each other after a game; each user has a karma score. 🎓 *Aggregations, and the real trade-off: compute karma on read (`AVG` query) vs denormalise into a column updated on write vs a scheduled rollup. Implement one, write down why.* Adds: `ratings` (rater_id, rated_id, game_id, score) + `karma` on `users`.

Module 8 — Geolocation / nearby games

"Games within N km of me." Requires PostGIS. 🎓 *Geospatial columns, distance/radius queries, spatial indexes (GiST), `ST_DWithin`.* Adds: PostGIS `geography` point on `venues`; `GET /games?lat=&lng=&radius_km=`.

Module 9 — Notifications (email + push)

On join / booking / payment, notify the relevant users. Start with FastAPI `BackgroundTasks`, then graduate to a Redis-backed queue (Celery or RQ). 🎓 *Background tasks, async I/O, job queues, retries/backoff, third-party senders (SendGrid for email, FCM for push).* Adds: `notifications`, `device_tokens`.

Module 10 — Teams (persistent)

Real teams with members and roles (captain/member); a team can host games. 🎓 *Many-to-many relationships, membership roles, modelling an entity vs a relationship.* Adds: `teams`, `team_members` (team_id, user_id, role).

Module 11 — Social graph (follow)

Follow other players; a feed of games hosted by people you follow. 🎓 *Self-referential many-to-many, feed-generation queries.* Adds: `follows` (follower_id, followee_id).

Module 12 — Admin & RBAC

Admin-only endpoints (manage venues, cancel games, view all bookings) gated by role. 🎓 *Role-based access control, permission dependencies / scopes, an auth dependency that enforces roles.* Optionally mount SQLAlchemyAdmin.

4. Cross-cutting backend craft (apply in every module)

Not separate modules — habits to practise throughout. These are half the point.

- **Migrations:** an Alembic revision for every model change.
- **Tests:** at least one pytest per endpoint (happy path + the key edge case) using `httpx AsyncClient`.

- **Pagination:** every list endpoint (`limit`/`offset` or cursor).
 - **Dependency injection:** `get_db`, `get_current_user`, pagination params, role guards — all as dependencies.
 - **Error handling:** custom exception handlers; consistent error response shape; correct status codes.
 - **Config:** all secrets/URLs via Pydantic settings + `.env`, never hardcoded.
 - **Logging:** structured logs on requests and errors.
 - **Rate limiting:** `slowapi` on auth + write endpoints.
 - **Caching:** Redis-cache a hot read (e.g. the games feed) and learn cache invalidation.
 - **Docker:** everything runs via `docker-compose up`.
-

5. Build order

Strictly sequential. Finish, test, and commit each before the next:

```
0 setup → 1 auth → 2 games loop → 3 venues → 4 bookings → 5 payments → 6 chat → 7 ratings  
→ 8 geo → 9 notifications → 10 teams → 11 social → 12 admin
```

Modules 0–2 are the foundation everything else hangs off — get them solid first. After that the order is flexible, but never start a module before the current one passes its tests.

6. Working instructions (read this — it's a learning project)

- **You write the code; the agent assists.** The point is to learn FastAPI, so write each endpoint yourself. Use the agent to explain concepts, review your code, suggest the idiomatic approach, and unblock you when stuck — not to autocomplete whole modules. If the agent writes it, you didn't learn it.
- Build one module at a time, front-to-back, then commit. Never scaffold multiple modules at once.
- Every schema change → an Alembic migration. Every endpoint → a test.
- Import schemas from `models.py`; extend it per module, don't duplicate model classes.
- Keep derived values (open slots, status flips) computed in one place — never store drift-prone counters.
- When something is ambiguous, ask rather than improvising scope.